(CRLs) or running an *online certificate status protocol* (OCSP) service.

- Certificates must be automatically renewed periodically to prevent them from expiring. Because revocation involves blacklisting a certificate until it expires, short expiry times are preferred to prevent CRLs becoming too large. Ideally certificate renewal should be completely automated.

Using an intermediate CA

Directly issuing certificates from the root CA trusted by all your microservices is simple, but in a production environment you'll want to automate issuing certificates. This means that the CA needs to be an online service responding to requests for new certificates. Any online service can potentially be compromised, and if this is the root of trust for all TLS certificates in your cluster (or many clusters) then you'd have no choice in this case but to rebuild the cluster from scratch. To improve the security of your clusters you can instead keep your root CA keys offline and only use them to periodically sign an *intermediate CA* certificate. This intermediate CA is then used to issue certificates to individual microservices. If the intermediate CA is ever compromised, you can use the root CA to revoke its certificate and issue a new one. The root CA certificate can then be very long lived, while intermediate CA certificates are changed regularly.

To get this to work, each service in the cluster must be configured to send the intermediate CA certificate to the client along with its own certificate, so that the client can construct a valid *certificate chain* from the service certificate back to the trusted root CA.

If you need to run multiple clusters, you can also use a separate intermediate CA for each cluster and use *name constraints* (https://tools.ietf.org/html/rfc5280#section-4.2.1.10) in the intermediate CA certificate to restrict which names it can issue certificates for (but not all clients support name constraints). Sharing a common root CA allows clusters to communicate with each other easily, while the separate intermediate CAs reduce the scope if a compromise occurs.

10.3.2 Using a service mesh for TLS

In a highly dynamic environment like Kubernetes, it is not advisable to attempt to run a PKI manually. There are a